Docker 应用部署-Redis

本文包含两部分：

- **第一部分（第 1~5 节）**：普通 Redis 部署（缓存/消息队列场景）
- **第二部分（第 6~7 节）**：Redis Stack 部署（AI向量数据库场景，含RediSearch向量检索能力）

---

1 拉取镜像

先演示安装普通Redis

```
docker pull redis:7.4.8
```

2 配置与数据目录

要在容器外部配置`Redis`，需要下载官方包，解压然后找到配置文件（`redis.conf`）并修改。

然后修改如下列的一些关键项：

```
bind 127.0.0.1 -::1 #注释掉这部分，这是限制redis只能本地访问
protected-mode no #默认yes，开启保护模式，会限制为本地访问
daemonize no #默认no，改为yes意为以守护进程方式启动，改为yes会使配置文件方式启动redis失败
databases 3 #数据库个数（可选）
dir ./ #输入本地redis数据库存放文件夹（可选）
user default on ~* +@all >01star #设置default用户的密码
appendonly yes #开启AOF持久化（可选）
```

提示：我资源里面已经提供了修改后的`redis.conf`配置文件

创建数据目录

```
mkdir -p /home/redis/redis-7.4.8/data/
```

然后将`redis.conf`配置文件上传到`/home/redis/redis-7.4.8`目录下面

3 启动服务

启动容器

```
docker run \
-id \
--name redis7 \
--restart always \
-p 6379:6379 \
-v /home/redis/redis-7.4.8/redis.conf:/etc/redis/redis.conf \
-v /home/redis/redis-7.4.8/data:/data \
redis:7.4.8 \
redis-server /etc/redis/redis.conf
```

参数说明：

- **-p `6379:6379`**
  - 将容器的 6379端口映射到宿主机的6379 端口
- **--restart always**
  - **--restart** 具体参数值详细信息：
    - no：容器退出时，不重启容器；
    - on-failure：只有在非0状态退出时才重新启动容器；
    - always：无论退出状态是如何，都重启容器；
- **-v**
  - 数据卷映射，主要映射数据目录和`Redis`服务器配置文件

4 连接`Redis`

```
# 进入docker容器，使用redis自带的客户端连接
docker exec -it redis7 redis-cli
```

5 防火墙端口

想要外部访问到你的数据库需要开放宿主机端口，我这里映射的是6379端口，所以开放6379端口。

注意：如果是云服务器，不需要通过下面命令开放端口，需要登录云控制台，配置防火墙入方向规则

```
#查看是否已经开放6379端口
firewall-cmd --list-port
#没有开放使用下面命令开放
firewall-cmd --add-port 6379/tcp --permanent
#重新加载防火墙
firewall-cmd --reload
```

---

6 RedisStack安装

6.1 为什么需要RedisStack?

普通 `Redis` 不包含向量检索能力，而 **Redis Stack** 在 Redis 基础上内置以下模块：

模块	能力	AI 场景
RediSearch	全文检索 + **向量相似度搜索**	核心！存储 embedding 并做 ANN 检索
RedisJSON	JSON 文档存储	存储结构化元数据
RedisTimeSeries	时序数据	监控/日志分析
RedisBloom	概率数据结构	去重/过滤

在 RAG（检索增强生成）流程中，**RediSearch 的向量检索** 是关键的召回环节：

```
文档 → Embedding 模型 → 向量 → Redis Stack (RediSearch)
                                          ↑
用户提问 → Embedding 模型 → 查询向量 → 相似度检索 → 召回 Top-K 片段 → LLM 生成回答
```

6.2 拉取镜像

```
docker pull redis/redis-stack:7.4.0-v8
```

版本说明：`7.4.0-v8` 是当前最新的 Redis Stack 稳定版，基于 Redis 7.4。
如需要特定版本，可访问 Docker Hub 查看所有 Tag。

6.3 配置与数据目录

Redis Stack 同样支持通过 `redis.conf` 进行自定义配置。可以从 官方仓库 获取配置文件模板，关键配置与普通 Redis 基本一致，额外建议：

```
# ========== 基础配置（与普通 Redis 相同） ==========
protected-mode no
# 设置密码（Redis Stack 使用 requirepass 而非 ACL）
requirepass 01star
dir ./
appendonly yes
# 最大内存
maxmemory 2gb
# 内存淘汰策略
maxmemory-policy allkeys-lru

# Redis Stack在启动时默认会加载所有模块，使用loadmodule指令显式加载模块
loadmodule /opt/redis-stack/lib/redisearch.so
loadmodule /opt/redis-stack/lib/rejson.so
```

创建数据目录：

```
mkdir -p /home/redis/redis-7.4.8-stack/data/
```

然后将 `redis.conf` 配置文件上传到 `/home/redis/redis-7.4.8-stack/` 目录下面，并重命名为 `redis-stack.conf`。

> **提示**：如果不需要自定义配置，Redis Stack 可以直接启动，它会使用内置默认配置。

6.4 启动容器

```
docker run \
-id \
--name redis7-stack \
--restart always \
-p 6380:6379 \
-p 8001:8001 \
-v /home/redis/redis-7.4.8-stack/redis-stack.conf:/redis-stack.conf \
-v /home/redis/redis-7.4.8-stack/data:/data \
redis/redis-stack:7.4.0-v8
```

参数说明：

- `-p 6380:6379`：将容器的 Redis 端口映射到宿主机的 `6380`，避免与前面普通 Redis 的 `6379` 冲突
- `-p 8001:8001`：映射 **RedisInsight** 可视化 Web 管理界面（默认端口 8001）
- `-v ... /redis-stack.conf:/etc/redis-stack.conf`：挂载自定义配置文件

> **注意**：如果宿主机没有运行普通 Redis，可以改回 `-p 6379:6379`。

6.5 连接RedisStack

```
# 进入容器，使用 redis-cli 连接（带密码认证）
docker exec -it redis7-stack redis-cli -a 01star

# 或者在容器内先进入 cli 再认证
docker exec -it redis7-stack redis-cli
127.0.0.1:6379> AUTH 01star
OK
```

6.6 防火墙端口

Redis Stack 额外暴露了 `6380`（Redis 端口）和 `8001`（RedisInsight Web 端口），需要在防火墙开放：

```
# 开放 Redis Stack 的 Redis 端口
firewall-cmd --add-port 6380/tcp --permanent
# 开放 RedisInsight Web 管理端口
firewall-cmd --add-port 8001/tcp --permanent
# 重新加载防火墙
firewall-cmd --reload
```

> **云服务器用户**：请登录云控制台，在安全组/防火墙规则中添加入方向 TCP:6380 和 TCP:8001。

---

7 验证向量检索能力

7.1 检查RediSearch模块

redis-cli中执行命令

```
127.0.0.1:6379> MODULE LIST
```

输出中应包含 `search`（RediSearch）、`ReJSON`、`bf`（RedisBloom）、`timeseries` 等模块。

7.2 创建向量索引

以 Cosine 相似度为例，创建一个维度为 1536 的向量索引（对应 OpenAI text-embedding-ada-002 的 embedding 维度）：

redis-cli中执行命令

```
# 创建索引：HNSW 算法 + Cosine 距离
FT.CREATE idx_docs ON HASH PREFIX 1 doc: SCHEMA content TEXT embedding VECTOR HNSW 6 TYPE FLOAT32 DIM 1536 DISTANCE_METRIC COSINE
```

参数说明：

参数	说明
`idx_docs`	索引名称
`ON HASH`	数据结构类型（HASH 或 JSON）
`PREFIX 1 doc:`	只索引 key 前缀为 `doc:` 的数据
`content TEXT`	普通文本字段
`VECTOR HNSW 6`	向量字段，使用 HNSW 算法，M=6
`TYPE FLOAT32`	向量值类型
`DIM 1536`	向量维度
`DISTANCE_METRIC COSINE`	相似度度量：COSINE / L2 / IP

7.3 插入向量数据

redis-cli中执行命令

```
# 插入一条文档（embedding 为 1536 维 float32，这里仅示意）
#HSET doc:1
#  content "Redis Stack 支持向量检索，可用于 RAG 场景"
#  embedding <此处填入实际的 1536 维 float32 向量>
# 示例命令
HSET doc:1 content "Redis Stack 支持向量检索，可用于 RAG 场景" embedding "\x00\x00\x80\x3f\x00\x00\x00\x40\x00\x00\x40\x40"
```

上述十六进制对应 3 个 float32 数：1.0, 2.0, 3.0。**实际使用需替换为真实 1536 维向量的十六进制串**（长度 1536×4=6144 字节）。

7.4 执行向量相似度检索

redis-cli中执行命令

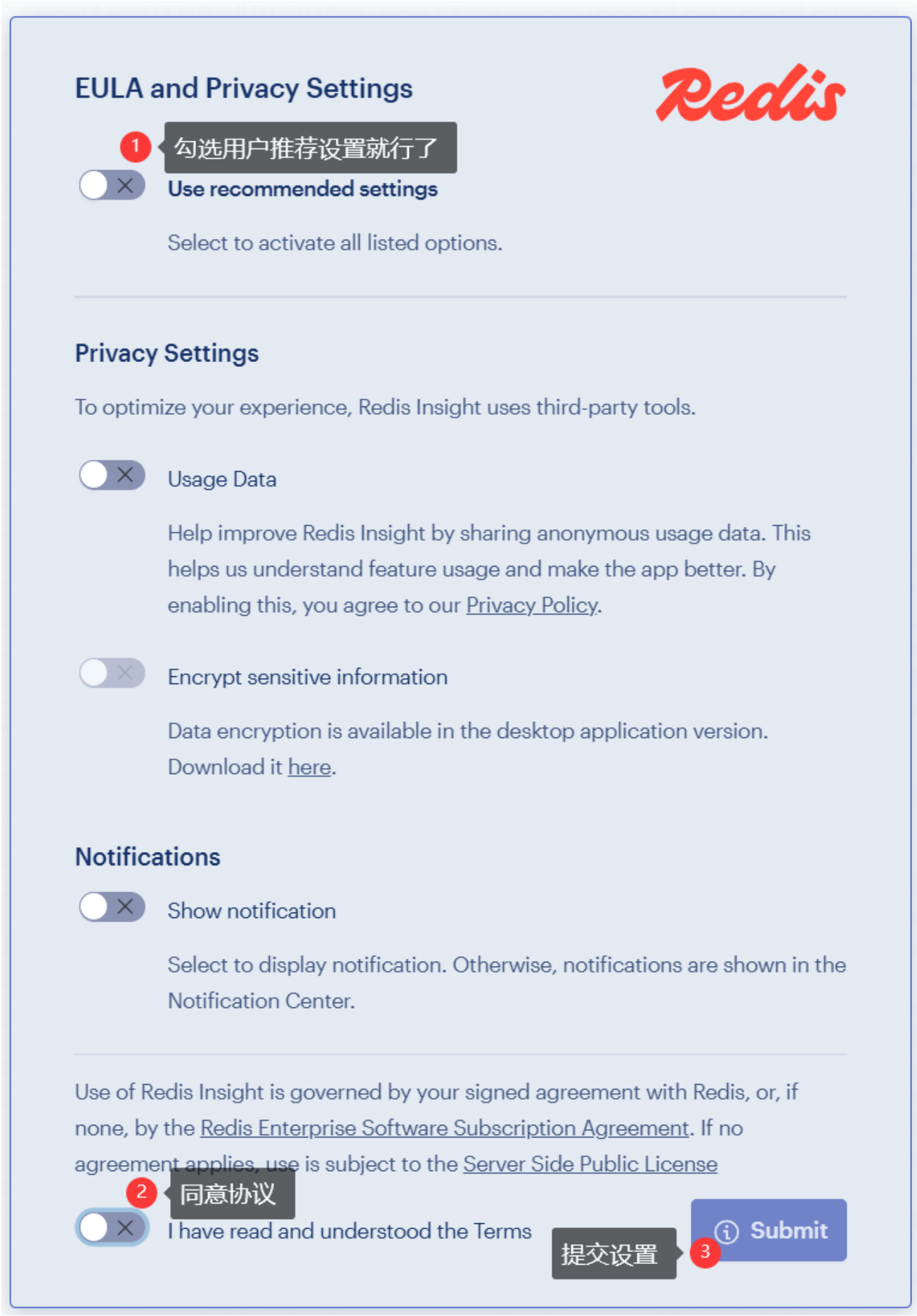
```
# 按向量相似度检索 Top-3，返回 content 字段和 score
#FT.SEARCH idx_docs
#  "*=>[KNN 3 @embedding $query_vec AS score]"
#  PARAMS 2 query_vec <此处填入查询向量>
#  SORTBY score
#  DIALECT 4
#  RETURN 2 content score
# 示例命令
FT.SEARCH idx_docs "*=>[KNN 3 @embedding $query_vec AS score]" PARAMS 2 query_vec "\x00\x00\x80\x3f\x00\x00\x00\x40\x00\x00\x40\x40" SORTBY score DIALECT 4 RETURN 2 content score
```

7.5 访问RedisInsight

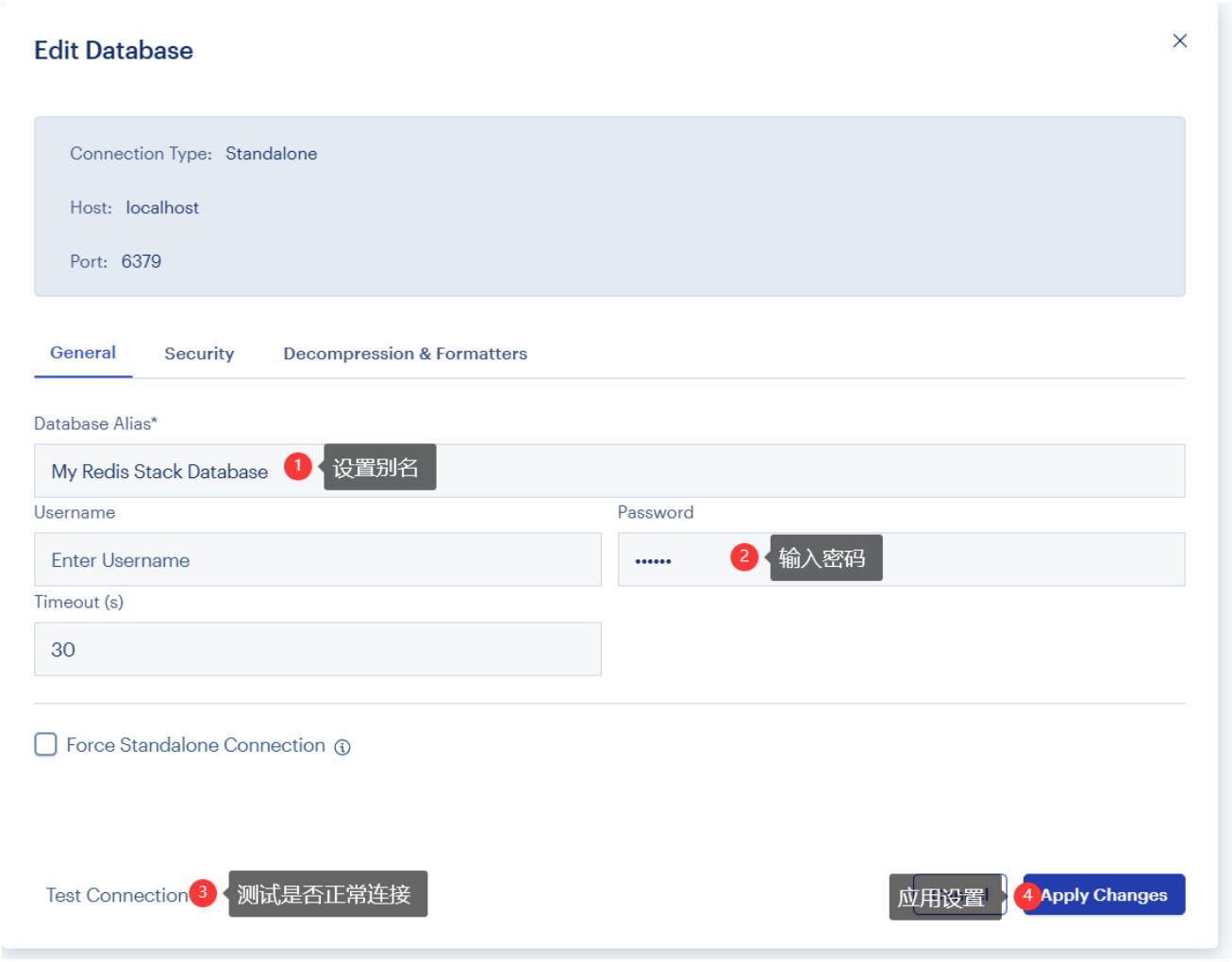
浏览器访问 `http://<服务器IP>:8001`，即可使用 RedisInsight 图形化管理工具：

- 可视化浏览 key 和数据
- 图形化创建索引
- 查询性能分析
- 向量搜索调试

首次启动后，你会看到如下界面



提交后，进入数据库设置界面



浏览可以看到之前创建的数据

